



Software Design Patterns Lab

CSE-3216

Assignment Report - 1

Integrating different implementations of the Singleton Pattern
in Python with a MongoDB Connection manager and evaluating
their behavior under different scenarios

Submitted by:

Md. Mahmudur Rahman Moin (AE-30)

Date of Submission:

April 22, 2026

Department of Computer Science and Engineering
University of Dhaka

1. Objectives

To evaluate six different Singleton implementations integrated with a MongoDB connection manager, specifically analyzing instance consistency, behavior under high concurrency, and performance overhead.

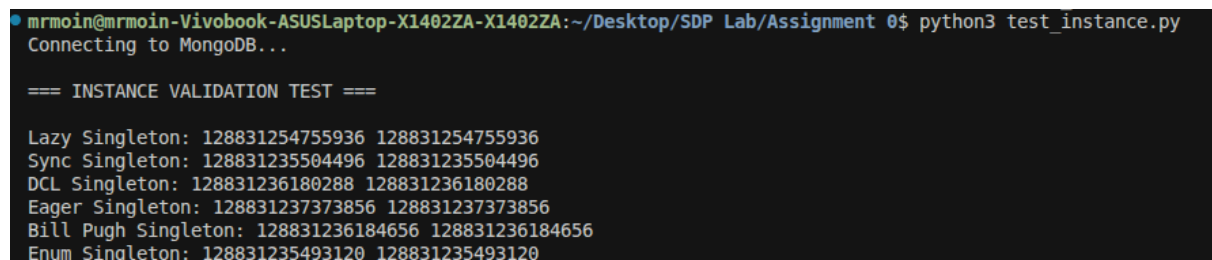
2. Instance Validation Results

This experiment verified whether multiple calls to the instantiation method return the same object reference in a single-threaded environment.

Observed Data (Sample IDs):

Implementation	Instance 1 ID	Instance 2 ID	Observation
Lazy Initialization	127158828464736	127158828464736	Consistent
Sync Method	127158809213872	127158809213872	Consistent
Double-Checked Locking	127158809872704	127158809872704	Consistent
Eager Initialization	127158811082704	127158811082704	Consistent
Bill Pugh Implementation	127158809877072	127158809877072	Consistent
Enum-based Singleton	127158809200528	127158809200528	Consistent

Table 1: Output table of instance validation test for different Singleton implementations



```

mrmoim@mrmoim-Vivobook-ASUSLaptop-X1402ZA-X1402ZA:~/Desktop/SDP Lab/Assignment 0$ python3 test_instance.py
Connecting to MongoDB...

=== INSTANCE VALIDATION TEST ===

Lazy Singleton: 128831254755936 128831254755936
Sync Singleton: 128831235504496 128831235504496
DCL Singleton: 128831236180288 128831236180288
Eager Singleton: 128831237373856 128831237373856
Bill Pugh Singleton: 128831236184656 128831236184656
Enum Singleton: 128831235493120 128831235493120

```

Figure 1: Instance Validation Test Result

2.1. Observations

Under sequential execution, all six implementations successfully maintained a single instance. The memory addresses for **Instance 1** and **Instance 2** were identical in every test case, confirming that the basic Singleton logic holds true when resource contention is absent.

3. Multithreading Results

To test thread safety, 10 threads were spawned to attempt simultaneous instantiation of the MongoDB manager.

Concurrent Execution Summary:

Implementation	Success Rate	Unique IDs Found	Conclusion
Lazy (Basic)	0%	7 to 10	Not Thread Safe
Sync Method	100%	1	Thread Safe
Double-Checked Locking	100%	1	Thread Safe
Eager Initialization	100%	1	Thread Safe
Bill Pugh Implementation	0% 20%	6 to 8	Not Thread Safe (Python)
Enum-based Singleton	100%	1	Thread Safe

Table 2: Output table of thread safety test for different Singleton implementations

```

mrmoim@mrmoim-Vivobook-ASUSLaptop-X1402ZA-X1402ZA:~/Desktop/SDP Lab/Assignment 0$ python3 test threading.py

LazySingleton (Testing Thread Safety)...
138542329480816
138542329480816
138542329483168
138542329490752
138542329490752
138542329490752
138542329479808
138542329481824
138542329482736
138542294192320
RESULT: LazySingleton is NOT Thread Safe! (7 instances created)

SyncSingleton (Testing Thread Safety)...
138542329484752
138542329484752
138542329484752
138542329484752
138542329484752
138542329484752
138542329484752
138542329484752
138542329484752
138542329484752
RESULT: SyncSingleton is Thread Safe.

DCLSingleton (Testing Thread Safety)...
138542329490176
138542329490176
138542329490176
138542329490176
138542329490176
138542329490176
138542329490176
138542329490176
138542329490176
138542329490176
RESULT: DCLSingleton is Thread Safe.

```

Figure 2: Thread Safety Test Result - 1

```

mrmoim@mrmoim-Vivobook-ASUSLaptop-X1402ZA-X1402ZA:~/Desktop/SDP Lab/Assignment 0$ python3 test threading.py
EagerSingleton (Testing Thread Safety)...
138542328841264
138542328841264
138542328841264
138542328841264
138542328841264
138542328841264
138542328841264
138542328841264
138542328841264
138542328841264
138542328841264
RESULT: EagerSingleton is Thread Safe.

BillPughSingleton (Testing Thread Safety)...
138542294197024
138542294197024
138542329483072
138542329483024
138542329483024
138542294193712
138542329488832
138542294190544
138542294284560
138542294274192
RESULT: BillPughSingleton is NOT Thread Safe! (8 instances created)

EnumSingleton (Testing Thread Safety)...
138542328843856
138542328843856
138542328843856
138542328843856
138542328843856
138542328843856
138542328843856
138542328843856
138542328843856
138542328843856
138542328843856
RESULT: EnumSingleton is Thread Safe.

```

Figure 3: Thread Safety Test Result - 2

4. Performance Measurements

Performance was measured across two metrics: **Initialization Time** (the very first call involving the MongoDB handshake) and **Access Time** (the total time for 100,000 subsequent retrievals).

Empirical Timing Data:

Implementation	Init Time (s)	Access Time (100,000 calls)
Eager Initialization	0.00000031	0.00422035 s
Enum-based	0.00000290	0.00904065 s
Lazy (Basic)	0.00108579	0.01329540 s
DCL (Lightweight)	0.00146397	0.01147315 s
Bill Pugh	0.00105568	0.01250748 s
Sync (Synchronized)	0.00161312	0.02119956 s

Table 3: Output table of Initialization and Access time test for different Singleton implementations

```

mrmoim@mrmoim-Vivobook-ASUSLaptop-X1402ZA-X1402ZA:~/Desktop/SDP Lab/Assignment 0$ python3 test_performance.py
Connecting to MongoDB...

=====
Implementation | Init Time (s) | Access Time (s)
-----
Lazy           | 0.00108579     | 0.01329540
Sync           | 0.00161312     | 0.02119956
DCL            | 0.00146397     | 0.01147315
Eager          | 0.00000031     | 0.00422035
Bill Pugh      | 0.00105568     | 0.01250748
Enum           | 0.00000290     | 0.00904065
=====

=== LIGHTWEIGHT VS SYNCHRONIZED COMPARISON ===
Synchronized Access Time: 0.021200s
DCL (Lightweight) Access Time: 0.011473s
Optimization: Double-Checked Locking is 45.88% faster than Synchronized.

```

Figure 4: Thread Safety Test Result - 1

4.1. Performance Analysis

1. **Access Overhead:** The Sync Method was the slowest during repeated access, taking twice as long as the DCL implementation. This is attributed to the lock acquisition overhead on every call.
2. **Lightweight Advantage:** DCL showed a 45.88% performance optimization over the Sync method by bypassing the lock after the initial creation.
3. **Initialization:** Eager and Enum showed near-zero initialization times because the resource (MongoDB client) was allocated at module-load/class-definition time.

5. Comparative Summary

1. **Fastest Implementation:** Eager Initialization provided the quickest access time and zero-latency initialization during the test phase. Enum-based followed closely as the fastest thread-safe lazy alternative.
2. **Thread Safety in Practice:** Sync, DCL, Eager, and Enum were the only implementations to successfully maintain a single MongoDB connection under high concurrency.
3. **Inconsistencies:** The Lazy (Basic) and Bill Pugh implementations showed significant failure in concurrent environments. In the case of the Lazy implementation, the MongoDB connection logic was triggered 7 times instead of 1, leading to resource duplication.
4. **Efficiency:** Double-Checked Locking (DCL) is the most efficient "Lazy" implementation, providing thread safety while remaining significantly faster than the standard Synchronized method.